Fuel Capacity Optimization System Documentation

> **Scope**: Complete documentation of the convergent fuel capacity optimization system, including iterative mission analysis, dynamic fuel load determination, and Key Performance Parameter (KPP) tracking for minimum required fuel capacity calculation.

Table of Contents

---

1) System Overview and Objectives

1.1 Purpose and Scope

The fuel capacity optimization system implements a sophisticated convergent iterative algorithm to determine the minimum required fuel capacity for mission completion. This approach replaces static, user-defined Maximum Take-Off Fuel (MTOF) values with dynamically optimized minimum fuel loads, eliminating superfluous fuel mass and improving aircraft performance through systematic optimization.

1.2 System Objectives

Primary Objectives:

- **Fuel Minimization**: Determine minimum required fuel capacity for mission completion
- **Convergent Optimization**: Implement iterative refinement until fuel load converges
- **Safety Integration**: Apply systematic safety buffers to optimized results
- **Performance Tracking**: Monitor Key Performance Parameters (KPPs) throughout optimization
- **Mission Integration**: Coordinate climb, cruise, and descent phase optimization

Key Components:

- Convergent iterative optimization loop
- Dynamic fuel load adjustment mechanism
- Multi-phase mission simulation integration
- Key Performance Parameter (KPP) evolution tracking
- Safety buffer application system
- Error handling and recovery mechanisms

1.3 System Flow Overview

The optimization system follows a systematic progression:

1. **Initialization**: Start with maximum fuel capacity as initial guess
2. **Mission Simulation**: Execute complete mission (climb + cruise + descent)
3. **Convergence Analysis**: Compare fuel consumed vs. initial fuel load
4. **Iteration Update**: Use consumed fuel as next iteration's initial load
5. **Convergence Detection**: Monitor relative fuel change until convergence
6. **Safety Application**: Apply safety buffer to converged result

---

2) Mathematical Foundation

2.1 Optimization Problem Formulation

Theory: The fuel optimization problem can be formulated as a constrained minimization problem seeking the minimum fuel capacity that ensures mission completion.

Mathematical Formulation:

$$\text{minimize: } F_{total} = F_{climb} + F_{cruise} + F_{descent}$$

Subject to:

$$\text{Mission completion constraints} \quad (1)$$
$$\text{Aircraft performance limits} \quad (2)$$
$$\text{Safety margin requirements} \quad (3)$$

Where:

- `F_total`: Total fuel consumption [kg]
- `F_climb`, `F_cruise`, `F_descent`: Fuel consumption in each mission phase [kg]

2.2 Convergence Algorithm Theory

Fixed-Point Iteration Scheme:

$$F_{k+1} = f(F_k)$$

Where:

- `F_k`: Initial fuel load at iteration k [kg]
- `f(F_k)`: Mission simulation function returning fuel consumed [kg]
- Convergence criterion: `|F_{k+1} - F_k| / F_k < ε`

Convergence Criteria:

$$\delta_{rel} = \frac{|F_{consumed,k} - F_{consumed,k-1}|}{F_{consumed,k-1}} < \varepsilon$$

Where ε = 0.001 (0.1% relative tolerance)

Safety Buffer Application:

$$F_{optimized} = F_{converged} \times (1 + \beta)$$

Where β = 0.05 (5% safety margin)

2.3 Mass Evolution Physics

Aircraft Mass Evolution:

$$m(t) = m_0 - \int_0^t \dot{m}_{fuel}(\tau)\, d\tau$$

Initial Mass Composition:

$$m_0 = W_{OE} + W_{PL} + F_{initial}$$

Where:

- `m_0` : Initial aircraft mass [kg]
- `W_OE` : Operating empty weight [kg]
- `W_PL` : Payload weight [kg]
- `F_initial` : Initial fuel load [kg]
- `ṁ_fuel(t)` : Instantaneous fuel flow rate [kg/s]

---

3) System Architecture and Data Structures

3.1 System Parameters

Convergence Control Parameters:

```
CONVERGENCE_TOLERANCE_RELATIVE = 0.001  # 0.1% relative tolerance
SAFETY_BUFFER_PERCENT = 0.05           # 5% safety buffer
MAX_ITERATIONS = 10                    # Safety limit
```

Mission Configuration Parameters:

```
TARGET_ALT_M = 10000.0                # Target cruise altitude [m]
START_ALTITUDE_M = 10.0               # Initial climb altitude [m]
CRUISE_DISTANCE_KM = 1500.0          # Cruise distance [km]
TARGET_DESCENT_ALT_M = 300.0         # Target descent altitude [m]
TARGET_DESCENT_MACH = 0.25           # Target approach Mach [-]
```

3.2 Data Structures

Mission Iteration Results:

```python
@dataclass
class MissionIterationResults:
    """Results from a single mission iteration."""

    # Required fields (no defaults)
    iteration: int
    initial_fuel_kg: float
    initial_mass_kg: float
    fuel_consumed_kg: float
    convergence_delta_percent: float

    # Phase-wise results (no defaults)
    climb_result: MinFuelSchedule
    cruise_result: CruiseResults
    descent_result: DescentResults

    # Mission totals (no defaults)
    total_time_s: float
    total_distance_km: float

    # Key performance parameters (no defaults)
    final_weight_kg: float
    climb_fuel_kg: float
    cruise_fuel_kg: float
    descent_fuel_kg: float
    climb_time_s: float
    cruise_time_s: float
    descent_time_s: float

    # Optional fields with defaults (aerodynamic performance tracking)
    avg_lift_climb_N: float = 0.0
    avg_drag_climb_N: float = 0.0
    avg_lift_cruise_N: float = 0.0
    avg_drag_cruise_N: float = 0.0
    avg_lift_descent_N: float = 0.0
    avg_drag_descent_N: float = 0.0

    # L/D ratios
    avg_ld_climb: float = 0.0
```

```
    avg_ld_cruise: float = 0.0
    avg_ld_descent: float = 0.0

    # Thrust lever positions
    avg_lever_climb: float = 0.0
    avg_lever_cruise: float = 0.0
    avg_lever_descent: float = 0.0

    # Specific energy (J/kg)
    avg_specific_energy_climb_J_kg: float = 0.0
    avg_specific_energy_cruise_J_kg: float = 0.0
    avg_specific_energy_descent_J_kg: float = 0.0
```

Convergence History:

```python
@dataclass
class ConvergenceHistory:
    """Tracking structure for convergence analysis."""

    iterations: List[MissionIterationResults]

    def add_iteration(self, result: MissionIterationResults):
        """Add iteration result to history."""
        self.iterations.append(result)

    def get_last_two_iterations(self) -> Tuple[MissionIterationResults, MissionIterationResults]:
        """Get the last two iterations for convergence analysis."""
        if len(self.iterations) < 2:
            raise ValueError("Need at least 2 iterations for convergence analysis")
        return self.iterations[-2], self.iterations[-1]

    def is_converged(self) -> bool:
        """Check if convergence has been achieved."""
        if len(self.iterations) < 2:
            return False

        prev, curr = self.get_last_two_iterations()
        delta = curr.convergence_delta_percent

        return abs(delta) < (CONVERGENCE_TOLERANCE_RELATIVE * 100.0)
```

---

4) Convergence Algorithm Implementation

4.1 Optimization Loop Structure

Function: `optimize_fuel_capacity()`

Purpose: Main optimization loop to determine minimum required fuel capacity through iterative convergence.

Algorithm:

```python
def optimize_fuel_capacity(aero: PyAerodynamicsWrapper, eng: EngineWrapper,
                           M_grid: np.ndarray, H_plot: np.ndarray,
                           lever_samples: int = 50) -> Tuple[MissionIterationResults, ConvergenceHi
    """
    Main optimization loop to determine minimum required fuel capacity.

    Process:
    1. Start with MAX_FUEL_KG as initial guess
    2. Run full mission and record fuel consumed
    3. Use fuel consumed as new initial fuel for next iteration
    4. Repeat until convergence (fuel difference < 0.1%)
    5. Apply 5% safety buffer to final result
    """

    # Initialize with maximum fuel capacity
    initial_fuel_current_kg = MAX_FUEL_KG
    history = ConvergenceHistory()
    iteration_count = 0

    while iteration_count < MAX_ITERATIONS:
        iteration_count += 1

        # Calculate current total mass (empty weight + payload + fuel)
        current_total_mass = W_OE_KG + W_PL_KG + initial_fuel_current_kg

        # Run single mission iteration (with error handling)
        try:
            iteration_result = run_single_mission_iteration(
                initial_mass_kg=current_total_mass,
                aero=aero, eng=eng, M_grid=M_grid, H_plot=H_plot,
                lever_samples=lever_samples, print_progress=True
            )
        except RuntimeError as e:
            # Handle mission failure with binary search recovery
            if "No feasible path" in str(e):
                if len(history.iterations) > 0:
                    last_successful = history.iterations[-1].initial_fuel_kg
                    initial_fuel_current_kg = (last_successful + initial_fuel_current_kg) / 2.0
                    continue
```

```
                else:
                    raise

        # Store iteration results and compute convergence delta
        iteration_result.iteration = iteration_count
        if iteration_count > 1:
            prev_result = history.iterations[-1]
            delta_kg = iteration_result.fuel_consumed_kg - prev_result.fuel_consumed_kg
            delta_percent = (delta_kg / prev_result.fuel_consumed_kg) * 100.0
            iteration_result.convergence_delta_percent = delta_percent
        else:
            iteration_result.convergence_delta_percent = float('inf')

        # Add to history
        history.add_iteration(iteration_result)

        # Check convergence
        if history.is_converged():
            # Apply safety buffer
            optimized_fuel = iteration_result.fuel_consumed_kg * (1.0 + SAFETY_BUFFER_PERCENT)
            break

        # Update fuel for next iteration
        initial_fuel_current_kg = iteration_result.fuel_consumed_kg

    return history.iterations[-1], history
```

4.2 Convergence Detection Algorithm

Function: `is_converged()`

Purpose: Determine if the optimization has reached convergence based on relative fuel change.

Mathematical Implementation:

```python
def is_converged(self) -> bool:
    """Check if convergence has been achieved."""
    if len(self.iterations) < 2:
        return False

    prev, curr = self.get_last_two_iterations()
    delta = curr.convergence_delta_percent

    # Convergence criterion: |δ_rel| < 0.1%
    return abs(delta) < (CONVERGENCE_TOLERANCE_RELATIVE * 100.0)
```

Convergence Mathematics:

$$\text{Converged} = \begin{cases} \text{True} & \text{if } \left| \frac{F_{consumed,k} - F_{consumed,k-1}}{F_{consumed,k-1}} \right| < 0.001 \\ \text{False} & \text{otherwise} \end{cases}$$

4.3 Error Recovery Mechanism

Binary Search Recovery:

```python
def handle_mission_failure(error_message: str, history: ConvergenceHistory,
                           current_fuel: float) -> float:
    """Handle mission failure with binary search recovery."""

    if "No feasible path" in error_message:
        if len(history.iterations) > 0:
            last_successful_fuel = history.iterations[-1].initial_fuel_kg
            # Binary search: midpoint between last successful and current failed
            recovery_fuel = (last_successful_fuel + current_fuel) / 2.0
            return recovery_fuel
        else:
            raise RuntimeError("Mission impossible with MAX_FUEL_KG")
    else:
        raise RuntimeError(f"Unhandled mission failure: {error_message}")
```

---

5) Mission Physics Integration

5.1 Single Mission Iteration Implementation

Function: `run_single_mission_iteration()`

Purpose: Execute a complete mission iteration (climb + cruise + descent) with dynamic mass tracking.

Algorithm:

```python
def run_single_mission_iteration(initial_mass_kg: float, aero: PyAerodynamicsWrapper,
                                 eng: EngineWrapper, M_grid: np.ndarray, H_plot: np.ndarray,
                                 lever_samples: int, print_progress: bool = True) -> MissionIterati
    """
    Execute a complete mission iteration (climb + cruise + descent).

    Args:
        initial_mass_kg: Initial aircraft mass for this iteration
        aero: Aerodynamics wrapper instance
        eng: Engine wrapper instance
        M_grid: Mach grid for optimization
        H_plot: Altitude grid for plotting
        lever_samples: Number of lever samples for DP optimization
        print_progress: Whether to print progress messages

    Returns:
        MissionIterationResults containing all phase results
    """

    atmospheric_props = AtmosphericProperties()

    # ========= CLIMB PHASE ========================================
    # Calculate starting Mach from takeoff velocity at start altitude
    a = atmospheric_props.a_from_altitude(START_ALTITUDE_M)
    start_mach = START_VELOCITY_MS / a

    # Create uniform altitude steps
    uniform_step_size = TARGET_ALT_M / len(H_plot)
    H_sched = np.arange(START_ALTITUDE_M, TARGET_ALT_M + uniform_step_size, uniform_step_size)

    # Solve 3D DP for climb
    dp_sched, dp_info = ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass(
        aero, eng, M_grid, H_sched,
        lever_samples=lever_samples,
        target_mach=TARGET_MACH,
        target_mach_tolerance=TARGET_MACH_TOLERANCE,
        start_mach=start_mach,
        start_lever=START_LEVER,
        mass_kg=initial_mass_kg  # Pass current mass for this iteration
```

```
    )

    climb_fuel = float(np.nan_to_num(dp_sched.cumFuel_kg, nan=0.0)[-1])
    climb_time_s = float(np.sum(np.nan_to_num(dp_sched.dt_s, nan=0.0)))

    # ========= CRUISE PHASE ========================================
    cruise_results = run_cruise_simulation(
        climb_result=dp_sched,
        initial_mass_kg=initial_mass_kg,
        target_distance_km=CRUISE_DISTANCE_KM,
        aero=aero, engine=eng,
        time_step_s=CRUISE_TIME_STEP_S,
        create_plots=False  # No plots during iteration
    )

    cruise_fuel = cruise_results.total_fuel_consumed_kg
    cruise_time_s = cruise_results.total_time_s

    # ========= DESCENT PHASE =======================================
    H_descent = np.linspace(cruise_results.altitude_m[-1], TARGET_DESCENT_ALT_M, N_ALTITUDE_STEPS)
    M_min_descent = max(0.2, TARGET_DESCENT_MACH - 0.1)
    M_max_descent = min(0.85, cruise_results.mach_number[-1] + 0.05)
    M_grid_descent = np.linspace(M_min_descent, M_max_descent, N_MACH_SAMPLES)

    descent_result, descent_info = run_descent_dp_optimization(
        cruise_results=cruise_results, climb_fuel_kg=climb_fuel, climb_time_s=climb_time_s,
        aero=aero, engine=eng, target_altitude_m=TARGET_DESCENT_ALT_M,
        target_mach=TARGET_DESCENT_MACH, n_altitude_steps=N_ALTITUDE_STEPS,
        n_mach_samples=N_MACH_SAMPLES, lever_samples=N_LEVER_SAMPLES
    )

    descent_fuel = descent_result.total_fuel_consumed_kg
    descent_time_s = descent_result.total_time_s

    # ========= COMPUTE SUMMARY =====================================
    total_fuel = climb_fuel + cruise_fuel + descent_fuel
    total_time_s = climb_time_s + cruise_time_s + descent_time_s

    # Calculate total mission distance using actual calculated values from each phase
    total_distance_km = calculate_mission_distance(dp_sched, cruise_results, descent_result, atmos
```

```
    final_weight = descent_result.final_weight_kg

    # Calculate performance metrics
    performance_metrics = calculate_performance_metrics(dp_sched, cruise_results, descent_result,
                                                        initial_mass_kg, aero, atmospheric_props)

    return MissionIterationResults(
        iteration=-1,  # Will be set by caller
        initial_fuel_kg=initial_mass_kg - W_OE_KG - W_PL_KG,
        initial_mass_kg=initial_mass_kg,
        fuel_consumed_kg=total_fuel,
        convergence_delta_percent=0.0,  # Will be computed by caller
        climb_result=dp_sched,
        cruise_result=cruise_results,
        descent_result=descent_result,
        total_time_s=total_time_s,
        total_distance_km=total_distance_km,
        final_weight_kg=final_weight,
        climb_fuel_kg=climb_fuel,
        cruise_fuel_kg=cruise_fuel,
        descent_fuel_kg=descent_fuel,
        climb_time_s=climb_time_s,
        cruise_time_s=cruise_time_s,
        descent_time_s=descent_time_s,
        **performance_metrics  # Unpack performance metrics
    )
```

5.2 Phase-Specific Physics Models

Climb Phase Integration:

$$J^*(h, M, \lambda) = \min_u \{L(h, M, \lambda, u) + J^*(h', M', \lambda')\}$$

Cruise Phase Integration:

$$T = D \quad \text{(thrust equals drag)} \tag{4}$$

$$L = W \quad \text{(lift equals weight)} \tag{5}$$

$$\dot{m}_{fuel} = \text{TSFC} \times T_{total} \tag{6}$$

Descent Phase Integration:

$$P_s = \frac{(T_{total} - D) \times V}{W} < 0 \quad \text{(energy dissipation)}$$

5.3 Dynamic Mass Tracking

Mass Evolution Throughout Mission:

```python
def track_dynamic_mass(initial_mass_kg: float, fuel_consumed_kg: float) -> float:
    """Track aircraft mass evolution during mission."""
    return initial_mass_kg - fuel_consumed_kg
```

Weight-Dependent Calculations:

$$C_L = \frac{W}{0.5 \times \rho \times V^2 \times S} \tag{7}$$

$$C_{D_i} = \frac{C_L^2}{\pi \times AR \times e} \tag{8}$$

$$D_{total} = D_{parasitic} + D_{induced} \propto W^2 \tag{9}$$

---

6) Performance Metrics Calculation

6.1 Aerodynamic Efficiency Metrics

Function: `calculate_aerodynamic_metrics()`

Purpose: Calculate lift-to-drag ratios and aerodynamic performance for each mission phase.

Implementation:

```python
def calculate_aerodynamic_metrics(phase_result, aero: PyAerodynamicsWrapper,
                                  atmospheric_props: AtmosphericProperties,
                                  initial_mass_kg: float, final_mass_kg: float) -> Dict[str, float]
    """Calculate aerodynamic performance metrics for a mission phase."""

    # Calculate average lift (approximately equal to weight for steady flight)
    avg_weight_kg = (initial_mass_kg + final_mass_kg) / 2.0
    avg_lift_N = avg_weight_kg * 9.81

    # Calculate drag using aerodynamics wrapper
    drag_vals = []
    ld_vals = []

    for i in range(len(phase_result.alt_m)):
        # Get atmospheric properties
        _, _, rho = atmospheric_props.isa_properties(phase_result.alt_m[i])
        a = atmospheric_props.a_from_altitude(phase_result.alt_m[i])

        # Calculate drag coefficient and drag force
        weight_kg = np.interp(i, [0, len(phase_result.alt_m)-1], [initial_mass_kg, final_mass_kg])
        CD = aero.get_drag_coefficient(phase_result.mach[i], phase_result.alt_m[i], weight_kg)
        drag = CD * 0.5 * rho * (phase_result.mach[i] * a)**2 * aero.params['S_REF_M2']
        drag_vals.append(drag)

        # Calculate L/D ratio
        if drag > 0:
            ld_vals.append((weight_kg * 9.81) / drag)

    return {
        'avg_lift_N': avg_lift_N,
        'avg_drag_N': float(np.mean(drag_vals)) if drag_vals else 0.0,
        'avg_ld': float(np.mean(ld_vals)) if ld_vals else 0.0
    }
```

6.2 Engine Performance Metrics

Function: `calculate_engine_metrics()`

Purpose: Calculate thrust lever positions and engine utilization metrics.

Implementation:

```
def calculate_engine_metrics(phase_result) -> Dict[str, float]:
    """Calculate engine performance metrics for a mission phase."""

    if hasattr(phase_result, 'lever') and len(phase_result.lever) > 0:
        avg_lever = float(np.mean(phase_result.lever))
    else:
        avg_lever = 0.0

    return {
        'avg_lever': avg_lever,
        'max_lever': float(np.max(phase_result.lever)) if hasattr(phase_result, 'lever') else 0.0,
        'lever_utilization': avg_lever  # Fraction of maximum thrust used
    }
```

6.3 Energy Management Metrics

Function: `calculate_energy_metrics()`

Purpose: Calculate specific energy and energy height evolution.

Mathematical Foundation:

$$E_{specific} = E_{potential} + E_{kinetic} \tag{10}$$

$$E_{potential} = g \times h \tag{11}$$

$$E_{kinetic} = \frac{1}{2} \times V^2 \tag{12}$$

$$E_{total} = g \times h + \frac{1}{2} \times V^2 \quad [J/kg] \tag{13}$$

$$H_{energy} = \frac{E_{total}}{g} = h + \frac{V^2}{2g} \quad [m] \tag{14}$$

Implementation:

```
def calculate_energy_metrics(phase_result, atmospheric_props: AtmosphericProperties) -> Dict[str,
    """Calculate energy management metrics for a mission phase."""

    specific_energy_vals = []

    for i in range(len(phase_result.alt_m)):
        # Calculate true airspeed
        a = atmospheric_props.a_from_altitude(phase_result.alt_m[i])
        velocity_mps = phase_result.mach[i] * a

        # Calculate specific energy components
        pe = 9.81 * phase_result.alt_m[i]  # Potential energy [J/kg]
        ke = 0.5 * velocity_mps**2         # Kinetic energy [J/kg]
        specific_energy = pe + ke
        specific_energy_vals.append(specific_energy)

    return {
        'avg_specific_energy_J_kg': float(np.mean(specific_energy_vals)) if specific_energy_vals e
        'energy_height_m': float(np.mean(specific_energy_vals)) / 9.81 if specific_energy_vals els
    }
```

---

7) Distance and Energy Analysis

7.1 Mission Distance Calculation

Function: `calculate_mission_distance()`

Purpose: Calculate total mission distance using actual trajectory data from each phase.

Implementation:

```python
def calculate_mission_distance(climb_result, cruise_results, descent_result,
                               atmospheric_props: AtmosphericProperties) -> float:
    """Calculate total mission distance using actual calculated values from each phase."""

    # Climb distance: Calculate from velocity and time
    climb_distance_km = 0.0
    if hasattr(climb_result, 'mach') and hasattr(climb_result, 'alt_m') and hasattr(climb_result,
        climb_distance_m = 0.0
        for i in range(len(climb_result.dt_s)):
            if i < len(climb_result.mach) and i < len(climb_result.alt_m):
                # Calculate true airspeed at each point
                a = atmospheric_props.a_from_altitude(climb_result.alt_m[i])
                velocity_mps = climb_result.mach[i] * a
                # Add horizontal distance for this time step
                climb_distance_m += velocity_mps * climb_result.dt_s[i]
        climb_distance_km = climb_distance_m / 1000.0

    # Cruise distance: Use actual distance from cruise results
    cruise_distance_km = 0.0
    if hasattr(cruise_results, 'distance_km') and len(cruise_results.distance_km) > 0:
        cruise_distance_km = float(cruise_results.distance_km[-1])
    else:
        cruise_distance_km = CRUISE_DISTANCE_KM  # Fallback

    # Descent distance: Calculate from velocity and time
    descent_distance_km = 0.0
    if hasattr(descent_result, 'mach') and hasattr(descent_result, 'alt_m') and hasattr(descent_re
        descent_distance_m = 0.0
        for i in range(len(descent_result.dt_s)):
            if i < len(descent_result.mach) and i < len(descent_result.alt_m):
                # Calculate true airspeed at each point
                a = atmospheric_props.a_from_altitude(descent_result.alt_m[i])
                velocity_mps = descent_result.mach[i] * a
                # Add horizontal distance for this time step
                descent_distance_m += velocity_mps * descent_result.dt_s[i]
        descent_distance_km = descent_distance_m / 1000.0

    return climb_distance_km + cruise_distance_km + descent_distance_km
```

7.2 Distance Integration Mathematics

Climb Distance Integration:

$$D_{climb} = \int_0^{t_{climb}} V(t)\,dt = \sum_{i=1}^{n} V_i \times \Delta t_i$$

Cruise Distance Integration:

$$D_{cruise} = \int_0^{t_{cruise}} V_{cruise}\,dt$$

Descent Distance Integration:

$$D_{descent} = \int_0^{t_{descent}} V(t)\,dt = \sum_{i=1}^{n} V_i \times \Delta t_i$$

Where:

- `V_i = M_i × a_i`: True airspeed at point i [m/s]
- `Δt_i`: Time step duration [s]
- `a_i`: Speed of sound at altitude i [m/s]

---

8) Code Execution Flow and Logic

8.1 System Entry Point and Initialization

Main Entry Function: Integration with fuel optimization system

Execution Sequence:

```
# 1. Fuel Optimization System Initialization
def initialize_fuel_optimization():
    """Initialize fuel optimization system parameters and state."""

    # Set up optimization parameters
    optimization_params = {
        'convergence_tolerance': CONVERGENCE_TOLERANCE_RELATIVE,
        'safety_buffer': SAFETY_BUFFER_PERCENT,
        'max_iterations': MAX_ITERATIONS,
        'initial_fuel_guess': MAX_FUEL_KG
    }

    # Set up mission parameters
    mission_params = {
        'target_altitude': TARGET_ALT_M,
        'cruise_distance': CRUISE_DISTANCE_KM,
        'target_descent_altitude': TARGET_DESCENT_ALT_M,
        'target_mach': TARGET_MACH
    }

    return optimization_params, mission_params
```

8.2 Optimization Execution Flow

Function: `execute_optimization_loop()`

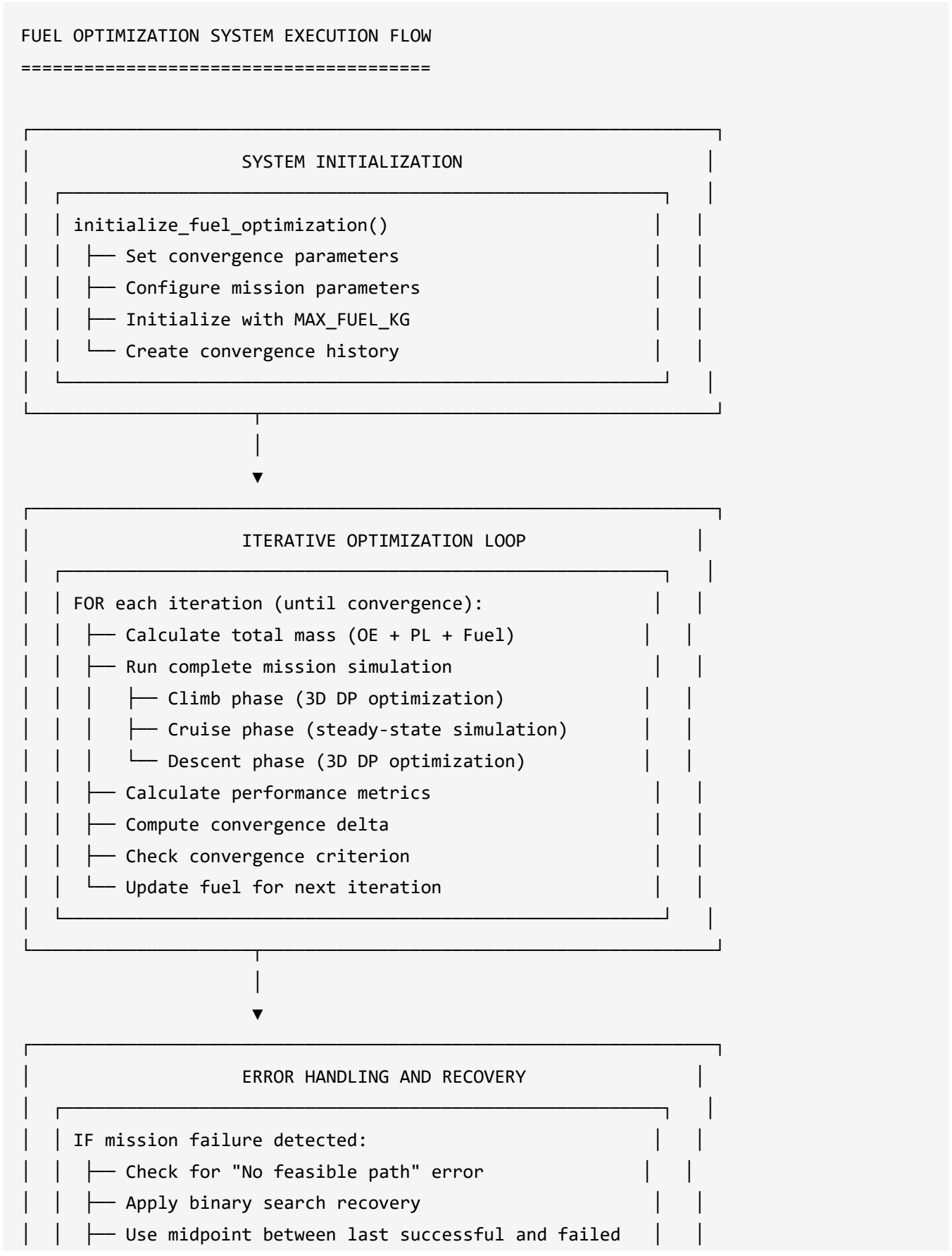
Step-by-Step Execution Flow:

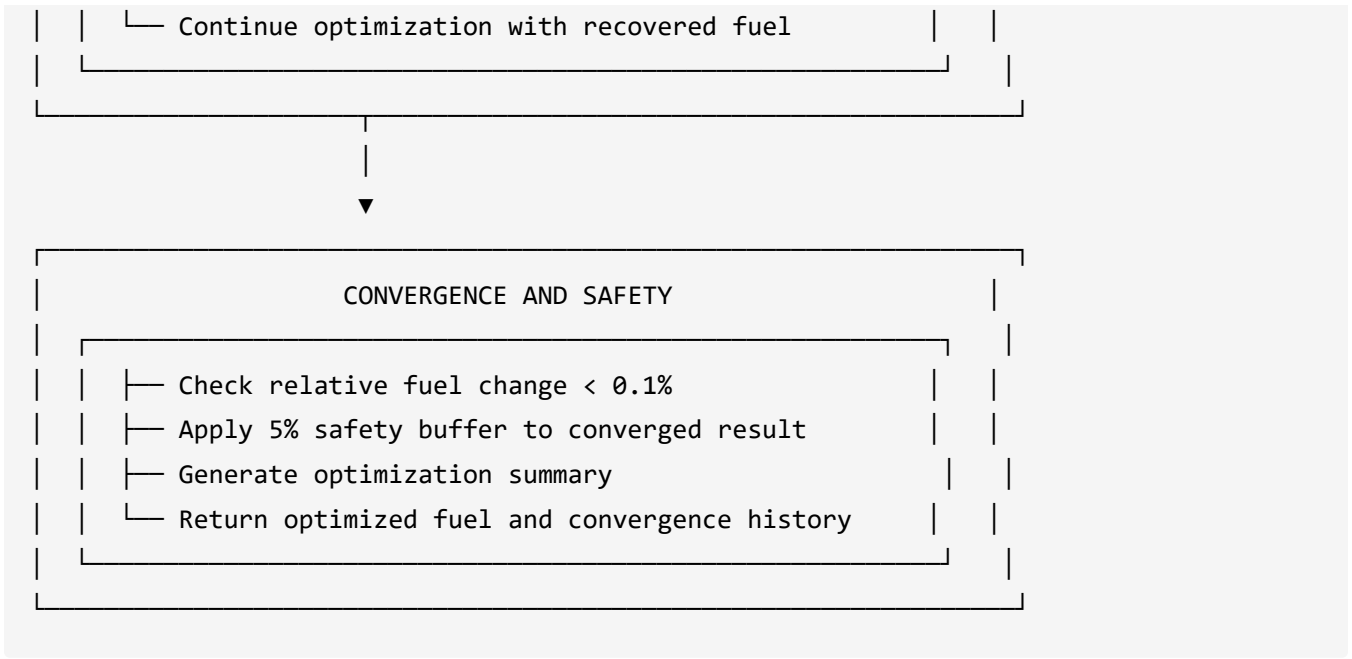
```
# PHASE 1: INITIALIZATION
def execute_optimization_loop():

    # Step 1: Initialize optimization parameters
    optimization_params, mission_params = initialize_fuel_optimization()

    # Step 2: Set up initial conditions
    initial_fuel_kg = optimization_params['initial_fuel_guess']
    history = ConvergenceHistory()
    iteration_count = 0
```

```
# PHASE 2: ITERATIVE OPTIMIZATION LOOP
    while iteration_count < optimization_params['max_iterations']:
        iteration_count += 1

        # Step 3: Calculate current total mass
        current_total_mass = W_OE_KG + W_PL_KG + initial_fuel_kg

        # Step 4: Execute mission simulation
        try:
            iteration_result = run_single_mission_iteration(
                initial_mass_kg=current_total_mass,
                aero=aero, eng=eng, M_grid=M_grid, H_plot=H_plot,
                lever_samples=lever_samples, print_progress=True
            )
        except RuntimeError as e:
            # Step 5: Handle mission failure
            if "No feasible path" in str(e):
                initial_fuel_kg = handle_mission_failure(str(e), history, initial_fuel_kg)
                continue
            else:
                raise

        # Step 6: Process iteration results
        iteration_result.iteration = iteration_count
        if iteration_count > 1:
            prev_result = history.iterations[-1]
            delta_kg = iteration_result.fuel_consumed_kg - prev_result.fuel_consumed_kg
            delta_percent = (delta_kg / prev_result.fuel_consumed_kg) * 100.0
            iteration_result.convergence_delta_percent = delta_percent
        else:
            iteration_result.convergence_delta_percent = float('inf')

        # Step 7: Add to convergence history
        history.add_iteration(iteration_result)
```

```
# PHASE 3: CONVERGENCE DETECTION AND COMPLETION
        # Step 8: Check convergence
        if history.is_converged():
            # Step 9: Apply safety buffer
            optimized_fuel = iteration_result.fuel_consumed_kg * (1.0 + optimization_params['safet
            break

        # Step 10: Update fuel for next iteration
        initial_fuel_kg = iteration_result.fuel_consumed_kg

        # Step 11: Safety checks
        if initial_fuel_kg < W_OE_KG * 0.1:
            initial_fuel_kg = W_OE_KG * 0.1  # Minimum fuel threshold

    # Step 12: Return optimization results
    return history.iterations[-1], history
```

8.3 Visual Code Flow Diagram


FUEL OPTIMIZATION SYSTEM EXECUTION FLOW
SYSTEM INITIALIZATION
initialize_fuel_optimization()
Set convergence parameters
Configure mission parameters
Initialize with MAX_FUEL_KG
Create convergence history
ITERATIVE OPTIMIZATION LOOP
FOR each iteration (until convergence):
Calculate total mass (OE + PL + Fuel)
Run complete mission simulation
Climb phase (3D DP optimization)
Cruise phase (steady-state simulation)
Descent phase (3D DP optimization)
Calculate performance metrics
Compute convergence delta
Check convergence criterion
Update fuel for next iteration
ERROR HANDLING AND RECOVERY
IF mission failure detected:
Check for "No feasible path" error
Apply binary search recovery
Use midpoint between last successful and failed


└── Continue optimization with recovered fuel
CONVERGENCE AND SAFETY
├── Check relative fuel change < 0.1%
├── Apply 5% safety buffer to converged result
├── Generate optimization summary
└── Return optimized fuel and convergence history

8.4 Function Call Hierarchy

```
Fuel Capacity Optimization System
├── optimize_fuel_capacity()
│   ├── initialize_fuel_optimization()
│   ├── ConvergenceHistory()
│   └── WHILE not converged:
│       ├── run_single_mission_iteration()
│       │   ├── ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass()
│       │   ├── run_cruise_simulation()
│       │   ├── run_descent_dp_optimization()
│       │   ├── calculate_mission_distance()
│       │   └── calculate_performance_metrics()
│       │       ├── calculate_aerodynamic_metrics()
│       │       ├── calculate_engine_metrics()
│       │       └── calculate_energy_metrics()
│       ├── history.add_iteration()
│       ├── history.is_converged()
│       └── handle_mission_failure() [if needed]
│
└── visualize_convergence_analysis()
    ├── plot_convergence_trajectory()
    ├── plot_kpp_evolution()
    ├── plot_optimization_comparison()
    ├── plot_aerodynamic_performance_analysis()
    ├── plot_3d_trajectory_comparison()
    └── plot_specific_energy_evolution()
```

---

9) Integration and Interface

9.1 Main System Interface

Function: `run_fuel_optimization_analysis()`

Integration Process:

```python
def run_fuel_optimization_analysis(aero: PyAerodynamicsWrapper, eng: EngineWrapper,
                                   M_grid: np.ndarray, H_plot: np.ndarray,
                                   lever_samples: int = 50) -> Tuple[MissionIterationResults, Conve
    """Run complete fuel optimization analysis with visualization."""

    # Execute optimization
    optimal_result, convergence_history = optimize_fuel_capacity(
        aero=aero, eng=eng, M_grid=M_grid, H_plot=H_plot, lever_samples=lever_samples
    )

    # Generate convergence visualizations
    visualize_convergence_analysis(convergence_history, save_plots=True)

    # Calculate final optimized parameters
    optimized_fuel = optimal_result.fuel_consumed_kg * (1.0 + SAFETY_BUFFER_PERCENT)
    optimized_mass = W_OE_KG + W_PL_KG + optimized_fuel

    print(f"[OPTIMIZATION COMPLETE]")
    print(f"Optimized fuel capacity: {optimized_fuel:.1f} kg")
    print(f"Optimized total mass: {optimized_mass:.1f} kg")
    print(f"Fuel savings: {MAX_FUEL_KG - optimized_fuel:.1f} kg ({(MAX_FUEL_KG - optimized_fuel) /

    return optimal_result, convergence_history
```

9.2 Configuration Interface

Optimization Configuration:

```
def configure_optimization_system(convergence_tolerance: float = 0.001,
                                  safety_buffer: float = 0.05,
                                  max_iterations: int = 100) -> Dict[str, Any]:
    """Configure optimization system parameters."""

    config = {
        'convergence_tolerance_relative': convergence_tolerance,
        'safety_buffer_percent': safety_buffer,
        'max_iterations': max_iterations,
        'parameters': {
            'target_altitude_m': TARGET_ALT_M,
            'cruise_distance_km': CRUISE_DISTANCE_KM,
            'target_descent_altitude_m': TARGET_DESCENT_ALT_M,
            'target_mach': TARGET_MACH
        }
    }

    return config
```

9.3 Results Export Interface

Results Export:

```python
def export_optimization_results(optimal_result: MissionIterationResults,
                                convergence_history: ConvergenceHistory,
                                export_format: str = 'json') -> str:
    """Export optimization results to specified format."""

    results_data = {
        'optimization_summary': {
            'converged_fuel_kg': optimal_result.fuel_consumed_kg,
            'optimized_fuel_kg': optimal_result.fuel_consumed_kg * (1.0 + SAFETY_BUFFER_PERCENT),
            'iterations_to_convergence': len(convergence_history.iterations),
            'fuel_savings_kg': MAX_FUEL_KG - (optimal_result.fuel_consumed_kg * (1.0 + SAFETY_BUFF
            'fuel_savings_percent': ((MAX_FUEL_KG - (optimal_result.fuel_consumed_kg * (1.0 + SAFE
        },
        'mission_performance': {
            'total_time_hours': optimal_result.total_time_s / 3600.0,
            'total_distance_km': optimal_result.total_distance_km,
            'climb_fuel_kg': optimal_result.climb_fuel_kg,
            'cruise_fuel_kg': optimal_result.cruise_fuel_kg,
            'descent_fuel_kg': optimal_result.descent_fuel_kg
        },
        'convergence_history': [
            {
                'iteration': iter_result.iteration,
                'fuel_consumed_kg': iter_result.fuel_consumed_kg,
                'convergence_delta_percent': iter_result.convergence_delta_percent
            }
            for iter_result in convergence_history.iterations
        ]
    }

    if export_format == 'json':
        import json
        return json.dumps(results_data, indent=2)
    else:
        return str(results_data)
```

10) Validation and Quality Assurance

10.1 Optimization System Validation

Parameter Validation:

```
def validate_optimization_parameters() -> None:
    """Validate optimization system parameters."""

    # Validate convergence parameters
    if CONVERGENCE_TOLERANCE_RELATIVE <= 0:
        raise ValueError("Convergence tolerance must be positive")

    if SAFETY_BUFFER_PERCENT < 0:
        raise ValueError("Safety buffer must be non-negative")

    if MAX_ITERATIONS <= 0:
        raise ValueError("Maximum iterations must be positive")

    # Validate mission parameters
    if TARGET_ALT_M <= START_ALTITUDE_M:
        raise ValueError("Target altitude must be greater than start altitude")

    if CRUISE_DISTANCE_KM <= 0:
        raise ValueError("Cruise distance must be positive")

    if not (0.1 <= TARGET_MACH <= 0.9):
        raise ValueError("Target Mach must be between 0.1 and 0.9")
```

10.2 Convergence Monitoring

Convergence Quality Assessment:

```python
def assess_convergence_quality(convergence_history: ConvergenceHistory) -> Dict[str, float]:
    """Assess the quality of convergence achieved."""

    if len(convergence_history.iterations) < 2:
        return {'convergence_quality': 0.0}

    # Analyze convergence behavior
    deltas = [abs(iter_result.convergence_delta_percent)
              for iter_result in convergence_history.iterations[1:]]

    # Calculate convergence metrics
    final_delta = deltas[-1] if deltas else float('inf')
    convergence_rate = np.mean(np.diff(deltas)) if len(deltas) > 1 else 0.0
    oscillation_measure = np.std(deltas[-5:]) if len(deltas) >= 5 else 0.0

    # Quality assessment
    quality_metrics = {
        'final_convergence_delta_percent': final_delta,
        'convergence_rate': convergence_rate,
        'oscillation_measure': oscillation_measure,
        'iterations_to_convergence': len(convergence_history.iterations),
        'convergence_quality': 1.0 - min(final_delta / (CONVERGENCE_TOLERANCE_RELATIVE * 100), 1.0
    }

    return quality_metrics
```

10.3 Physical Validation

Mission Physics Validation:

```python
def validate_mission_physics(optimal_result: MissionIterationResults) -> Dict[str, bool]:
    """Validate physical consistency of optimized mission."""

    validation_results = {}

    # Energy conservation check
    initial_energy = optimal_result.initial_mass_kg * 9.81 * START_ALTITUDE_M
    final_energy = optimal_result.final_weight_kg * 9.81 * TARGET_DESCENT_ALT_M
    fuel_energy = optimal_result.fuel_consumed_kg * 43.0e6  # Approximate fuel energy content [J/k

    energy_balance_error = abs((initial_energy - final_energy) - fuel_energy) / fuel_energy
    validation_results['energy_conservation'] = energy_balance_error < 0.1  # 10% tolerance

    # Mass conservation check
    mass_balance_error = abs((optimal_result.initial_mass_kg - optimal_result.final_weight_kg) - o
    validation_results['mass_conservation'] = mass_balance_error < 1.0  # 1 kg tolerance

    # Performance envelope check
    validation_results['climb_feasible'] = optimal_result.climb_fuel_kg > 0
    validation_results['cruise_feasible'] = optimal_result.cruise_fuel_kg > 0
    validation_results['descent_feasible'] = optimal_result.descent_fuel_kg >= 0

    # Time consistency check
    total_time_check = (optimal_result.climb_time_s + optimal_result.cruise_time_s + optimal_resul
    validation_results['time_consistency'] = abs(total_time_check - optimal_result.total_time_s) <

    return validation_results
```

10.4 Error Handling and Recovery

Comprehensive Error Handling:

```python
def safe_optimization_execution(aero: PyAerodynamicsWrapper, eng: EngineWrapper,
                                M_grid: np.ndarray, H_plot: np.ndarray) -> Tuple[MissionIterationRe
    """Safe optimization execution with comprehensive error handling."""

    try:
        # Validate inputs
        validate_optimization_parameters()

        # Execute optimization
        optimal_result, convergence_history = optimize_fuel_capacity(
            aero=aero, eng=eng, M_grid=M_grid, H_plot=H_plot
        )

        # Validate results
        convergence_quality = assess_convergence_quality(convergence_history)
        physics_validation = validate_mission_physics(optimal_result)

        # Check validation results
        if convergence_quality['convergence_quality'] < 0.8:
            print(f"[WARNING] Poor convergence quality: {convergence_quality['convergence_quality'

        if not all(physics_validation.values()):
            print(f"[WARNING] Physics validation failed: {physics_validation}")

        return optimal_result, convergence_history

    except Exception as e:
        print(f"[ERROR] Optimization failed: {e}")
        # Return fallback results or re-raise
        raise
```

Conclusion

The fuel capacity optimization system provides a mathematically rigorous, physically accurate approach to determining minimum required fuel capacity through convergent iterative optimization. By integrating detailed mission physics with systematic convergence algorithms,

the system enables significant fuel savings while maintaining safety margins and mission reliability.

The comprehensive architecture supports various mission profiles and aircraft configurations, making it a versatile tool for both design and operational applications in aerospace engineering. The system's robust error handling, validation mechanisms, and performance tracking ensure reliable operation across diverse operational scenarios.